



Sistema de Gerenciamento de Alunos

Documentação Técnica Completa — Análise de arquitetura, módulos, fluxo e integração com Firebase em Python.

NÍVEL TCC

PYTHON

FIREBASE

Arquitetura do Sistema

O projeto adota uma **arquitetura híbrida** com interface gráfica, lógica de negócios, persistência local e integração com Firebase. A separação em módulos segue o princípio de **desacoplamento**, permitindo manutenção e evolução independentes.

`dados.py`

Armazenamento local e fallback

`funções.py`

Lógica central do sistema

`bancodedados.py`

Comunicação com Firebase

`beta2.py`

Interface gráfica (GUI)

bancodedados.py – Comunicação com Firebase



Este módulo gerencia a comunicação com o Firebase via **requisições HTTP**. O Firebase utiliza estrutura **JSON**, exigindo conversão com `json.dumps()`. A biblioteca `requests` é utilizada para todas as operações.

POST

Criação de dados

GET

Leitura de dados

PATCH

Atualização parcial

DELETE

Remoção de registros

Funções: Criar() e Apagar()

Criar()

01

Recebe `nome`, `notafinal` e `situacao`

02

Cria dicionário Python e converte para JSON

03

Envia para endpoint `Alunos/json` via POST

04

Recebe `status_code` e exibe confirmação

Apagar()

01

Executa GET para obter todos os registros

02

Filtra campo `'Alunos'` e percorre com `for`

03

Compara nomes com `upper()` para evitar erro de caixa

04

Ao encontrar, executa DELETE e exibe sucesso

Funções: `Alterar()` e `LigacaoComDadosSalvos()`

`Alterar()`

Função de maior complexidade lógica. Recebe uma **lista de parâmetros** que controla tipo da operação, valores novos e campos a alterar. Dependendo de `campo[0]`, pode alterar um campo, múltiplos campos ou substituir a estrutura completa. Utiliza **PATCH** para atualizar partes do registro.

`LigacaoComDadosSalvos()`

- Executa GET no Firebase
- Converte JSON para Python
- Itera e monta dicionário padronizado
- Retorna lista completa para uso local



beta2.py – Interface Gráfica

Camada de apresentação do sistema, construída com **customtkinter** para interface moderna. Inclui integração com **pygame** (áudio) e **tkvideo** (vídeo).



ChamarJanela()

Cria instância CTK, define tamanho, limites de redimensionamento, título e configura grid (layout).



CriarSubJanela()

Cria janela secundária com `CTKToplevel` e aplica `grab_set()` para foco.



Funções de Interface

Adicionar(), **Mostrar()**, **Consultar()**, **Alterar()** e **Apagar()** — preparam e exibem dados, coletam inputs e chamam a lógica.

Controle de Mídia – Midia()



A função **Midia()** controla áudio e vídeo simultaneamente na interface, demonstrando **controle de estado** dentro da GUI.

 **PLAY**

```
pygame.mixer.music.play() e  
video.play()
```

 **PAUSE**

```
pygame.mixer.music.pause()
```

 **STOP**

```
pygame.mixer.music.unpause()
```

funções.py e dados.py

funções.py – Lógica Central

Módulo responsável por controle de dados, validação de entrada, cálculos de média e atualização de status (**Aprovado/Reprovado**). Garante **consistência dos dados** em todas as operações.

dados.py – Persistência Local

Armazena dados locais como **fallback** quando o banco remoto não está disponível.

Recebe dados da interface → processa valores → atualiza lista principal → atualiza média global da turma

listadaTurma

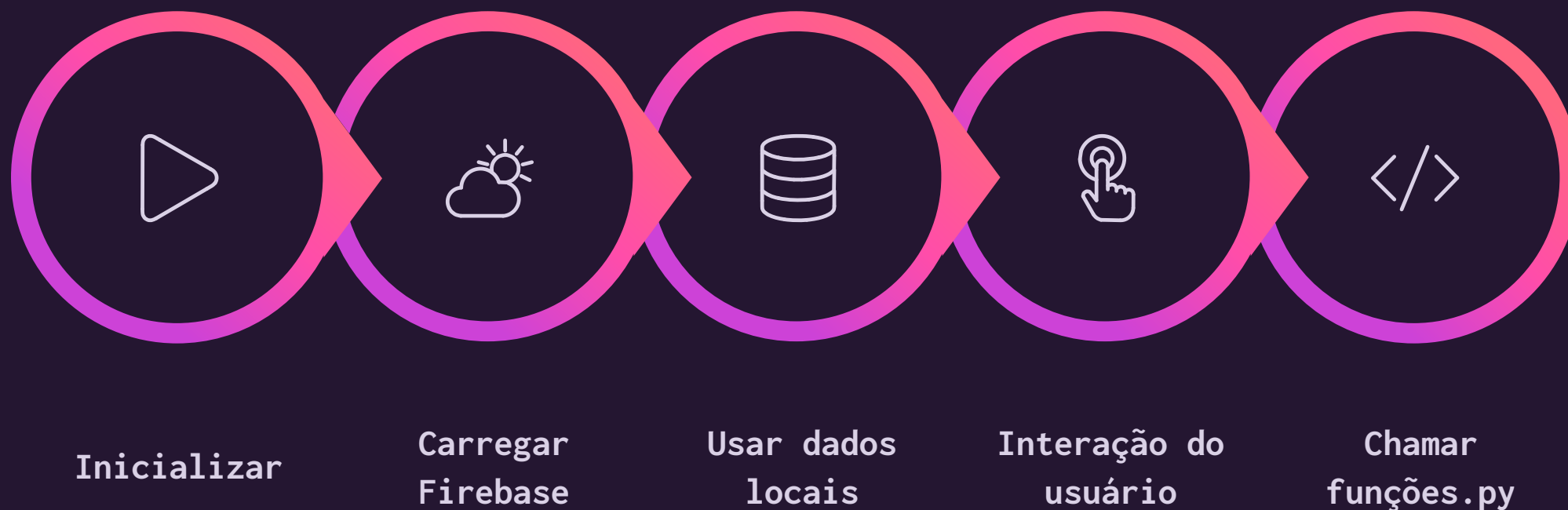
Lista de alunos

mediadaTurmaDefinitiva

Soma das médias

Fluxo Completo do Sistema

O fluxo demonstra a **integração completa** entre frontend, backend e banco de dados, desde a inicialização até o envio de alterações ao Firebase.



Esse fluxo garante que o sistema funcione tanto online (via Firebase) quanto offline (via dados locais), com validação centralizada no módulo `funções.py`.

Conclusão

O sistema apresenta características de **aplicações reais**, incluindo arquitetura modular, integração com API e interface gráfica moderna. Qualidade suficiente para **portfólio profissional** e base para trabalhos acadêmicos avançados.



Programação Estruturada



Interface Gráfica



APIs REST



Manipulação JSON



Arquitetura de Software